

STEP 8

Containerisation with Docker

Package the code, libraries, data and a freshly trained model into one image that runs anywhere.

The Dockerfile

```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt # cached until requirements change
COPY foodwaste/ api/ app/ data/food_waste_dataset.csv ...
RUN python -m foodwaste.train # trains in the image; fails below gate
EXPOSE 8000
HEALTHCHECK CMD python -c "urllib.request.urlopen('http://localhost:8000/health')"
CMD ["uvicorn", "api.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Key decisions

- **Train at build time.** The image always contains a model that passed the quality gate, built from the exact code and data in that image.
- **Layer order.** Requirements are installed before the code is copied, so editing code does not reinstall every library. That is why a rebuild takes about 11 seconds.
- **.dockerignore** keeps venv/, tests, notebooks, logs and old models out of the image.
- **HEALTHCHECK** lets Docker report the container as healthy only once the API answers.

docker compose: API + UI

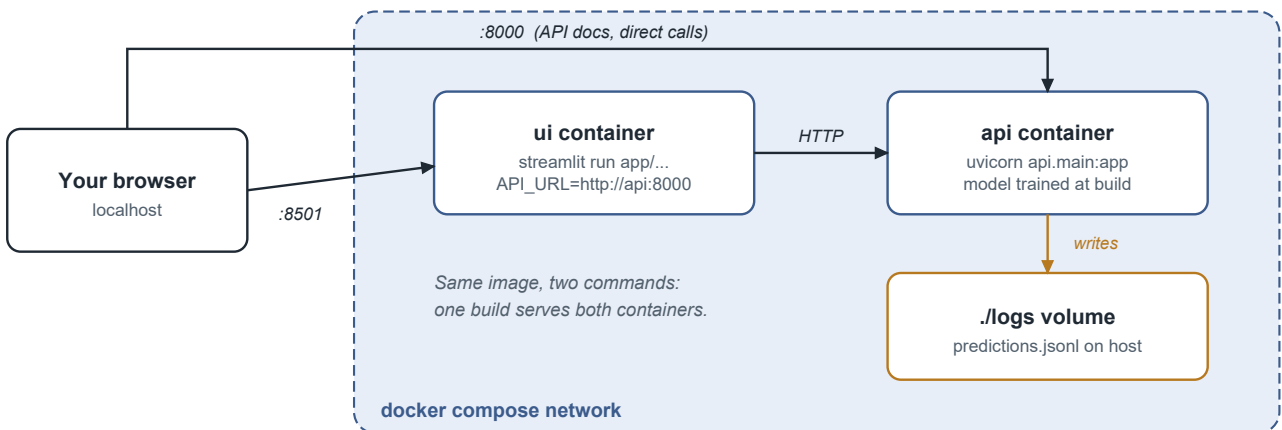


Figure 4 - One image, two containers. The UI reaches the API by its service name (http://api:8000). Logs are written to a folder on your machine.

```
docker compose up --build # first time, or after code changes
docker compose up        # later starts
docker compose down      # stop and remove containers
```

Then open the UI at <http://localhost:8501> and the API docs at <http://localhost:8000/docs>. In the UI, 'Served by: FastAPI service' confirms the two containers are talking.

Troubleshooting: port already in use

Error: 'ports are not available: listen tcp 0.0.0.0:8501: bind: Only one usage of each socket address...'

Cause: something on your machine already uses that port (here, the old IMDB Streamlit app on 8501 and a local uvicorn on 8000).

Fix: stop those processes (Ctrl+C in their terminals), then run `docker compose up` again.